Bluetooth:AWS IOT Core Integration with Gateway Firmware(Asset Tracker)

In this project, **Asset tracker V2** used as the base project with some more features added

1. Features :

- LTE/NB-IoT
- AWS IoT Configured
- MQTT Protocol between AWS IoT Core and Gateway via LTE
- Central Add its own parameters in Received packet from Peripheral and finally forward to Gateway(nrf9160) via BLE 5.0
- Gateway on Receiving Packet from Central forwards it to AWS IoT Core via LTE
- AWS Cloud, first read it to AWS IoT Core,then stores it to AWS Dynamodb through Lambda Function

2. Docklight Code Startup Data (UART0)

Code Startup Data

3. Docklight Data from Central (UART1)

Data from Central

4. Startup requirement:

- NCS SDK version 2.4.0
- VS code Studio
- Docklight
- NCS extension
- Github
- nrf9160DK/Custom Board
- AWS Cloud Account

5. Files included in this project:

- uart_1_event.c
- uart_1_event.h
- uart_1_module.c

6. Certificate

> Update the following way:
> AT+CFUN=4
> AT+CFIN=?
> Certificates must be updated via LTE link terminal to modem
> SEC TAG FOR AWS IS **123456**

AWS Cloud files	COPIED	NRF Modem
AmazonRootCA1(1)	TO	ca-cert

AWS Cloud files	COPIED	NRF Modem
4e52-private.pem	TO	private-key
4752-certificate.pem	TO	client-cert

7. Update the overlay-aws.conf file

```
CONFIG_AWS_FOTA=y
CONFIG_AWS_IOT_SEC_TAG=123456
CONFIG_AWS_IOT_BROKER_HOST_NAME="a30xxxxxxx-ats.iot.us-east-2.amazonaws.com"
CONFIG_AWS_FOTA_DOWNLOAD_SECURITY_TAG=123456
CONFIG_POSIX_MAX_FDS=24
CONFIG_AWS_IOT_CLIENT_ID_STATIC="nrf9160_thing"
          OR
#CONFIG_AWS_IOT_CLIENT_ID_APP=y
```

8. To clone the project: use the following directory

```
"C:\Nordic_apps\myapps"
and then create a new folder into it and in that folder download the repository
from GitHub
After that,
You have to add the build configurations to select the DK, in our case, we can use
nrf9160dk_nrf9160_ns
```

Configuration:

> prj.conf

Kconfig Fragments:

> overlay-aws.conf\

Extra-Cmake Argument

> -DOVERLAY_CONFIG=overlay-aws.conf

At the end

> PRESS Build Configuration

9. Code Explanation:

- **overlay-aws.conf**

> CONFIG_AWS_IOT_BROKER_HOST_NAME=""
> CONFIG_AWS_IOT_BROKER_HOST_NAME="a30695n0axlqin-ats.iot.us-east-2.amazonaws.com"

- **Prj.conf**

```
#CONFIG_NRF_CLOUD_MQTT=y
#CONFIG_CLOUD_CLIENT_ID_USE_CUSTOM=y
#CONFIG_NRF_CLOUD_SEC_TAG=16842753
```

- **src/cloud-certs/client-cert.pem**
- **src/cloud-certs/private-key.pem**

  Copy paste the files from the AWS cloud as described point # 6 above

- **src/cloud/cloud_codec/aws_iot/json_protocol_names_aws_iot.h**

```
#define TLV493D_DATA "tlv493d"
```

- **src/cloud/cloud_codec/cloud_codec.h**

```
-char * temperature;
+double temperature;
```

- **src/modules/sensor_module.c**

```
sensor_module_event->data.sensors.temperature = dummyDouble ;
```

10. AWS IoT Core Setting Images :

- Things
- Policy(A)
- Policy(B)
- Certificates
  - Go to settings and check policy and certificates both are active
  - Download the certificates
  - As Mentioned above
- Attach Certicates with Thing
- Attach Certicates with Policy
- MQTT End Point (AWS)
- MQTT End Point (FW)
- Topic Subscribe /MQTT Test (AWS Wild Card)
- Copy Certificates(Downloaded from AWS)
- Load AT Client FW
- Load Certificates to Modem
- Build Conf Gateway FW(A)
- Build Conf Gateway FW(B)

  Open COM3(Terminal Output) & COM4(Commands)

- MQTT Test Terminal (Data from NRFDK)
- MQTT.fx Broker Setting

11. AWS DynamoDB & Lambda Setting Images :

- Rule(A)
- Rule(B)

- Rule Lambda Fn(C)
- Rule Lambda Fn(D)
- Lambda Fn Conf(A)
- Lambda Fn Conf(B)
- Lambda Fn Conf(C)
- Table
- Table name Reference
- IAM Access Error
- IAM Access Role (A)
- IAM Access Role (B)
- IAM Access Permission (A)
- IAM Access Permission (B)
- IAM Access Permission (C)
- IAM Access Permission (D)
- Lambda Fn Test
- DynamoDB Table Explore Item
- Data Publish From Mqtt.fx
- Updated DynamoDB Table

12. FOTA AWS Setting Images:

- Bucket
- Bucket app_data.bin
- Bucket Job Doc
- Bucket Files Added
- Bucket name Job Doc
- Job (A)
- Job (B)
- Job (C)
- Job (D)

13. FOTA Started:

- FOTA Started
- FOTA Done!
- FW Updated

14.AWS IoT Core And DynamoDB Video Links:

AWS IoT Core

DynamoDB & Lambda Function

15.Lambda Fn Code:

```
import boto3

def lambda_handler(event, context):
    # Existing code to extract other parameters
```

```
    timestamp = str(event['state']['reported']['roam']['ts'])
    band = event['state']['reported']['roam']['v']['band']
    network = event['state']['reported']['roam']['v']['nw']
    rsrp = event['state']['reported']['roam']['v']['rsrp']
    area = event['state']['reported']['roam']['v']['area']
    mccmnc = event['state']['reported']['roam']['v']['mccmnc']
    cell = event['state']['reported']['roam']['v']['cell']
    ip = event['state']['reported']['roam']['v']['ip']
   # tlv493d = event['state']['reported']['roam']['v']['tlv493d']

    # Split the tlv493d values
    tlv493d_values = event['state']['reported']['roam']['v']['tlv493d'].split()
   # tlv493d_values_converted = [int(value, 16) for value in tlv493d_values]

       # Specific names for each of the 32 tlv493d values
    tlv493d_column_names = [
        'N1','N2', 'N3', 'N4','N5', 'N6', 'N7', 'N8','N9', 'N10', 'N11',
'N12','N13','N14','N15','N16','N17','N18','N19','N20',
        'N21','N22','N23','N24','N25','N26','N27','N28','N29','N30','N31','N32'
    ]


    # Check if there are 32 values, otherwise handle the error or adjust as needed
    if len(tlv493d_values) != 32:
        print("Error: tlv493d does not have 32 values.")
        return {
            'statusCode': 500,
            'body': 'Error: tlv493d does not have 32 values.'
        }

    # Preparing the item for DynamoDB
    item = {
        'timestamp': {'S': timestamp},
        'band': {'N': str(band)},
        'network': {'S': network},
        'rsrp': {'N': str(rsrp)},
        'area': {'N': str(area)},
        'mccmnc': {'N': str(mccmnc)},
        'cell': {'N': str(cell)},
        'ip': {'S': ip}
        #'tlv493d': {'S': tlv493d}
    }

    # Add tlv493d values to the item
    for i, value in enumerate(tlv493d_values):
        column_name = tlv493d_column_names[i]
        item[column_name] = {'S': value}

    # DynamoDB client
    client = boto3.client('dynamodb')

    # Storing the item in DynamoDB
    response = client.put_item(
        TableName='nrf9160_table',
```

```
            Item=item
        )

        return {
            'statusCode': 200,
            'body': 'Data processed successfully'
        }
```

16.MQTT Json Packet:

```
{
  "state": {
    "reported": {
      "roam": {
        "v": {
          "band": 12,
          "nw": "LTE-M",
          "rsrp": -76,
          "area": 36867,
          "mccmnc": 310410,
          "cell": 84690705,
          "ip": "10.165.223.207",
          "tlv493d": "B0 FC B2 B3 01 01 01 A8 24 75 25 84 59 99 99 FF FF AA AA AA
A9 41 3E 5E DC 01 01 86 FF FF FF FF "
        },
        "ts": 1709134814587
      }
    }
  }
}
```

Dynamodb Reference # 1

Dynamodb Reference # 2

AWS IoT Reference # 1

AWS IoT Reference # 2

AWS IoT Reference # 3

AWS IoT Topic Reference # 1

AWS IoT Job

AWS FOTA Job